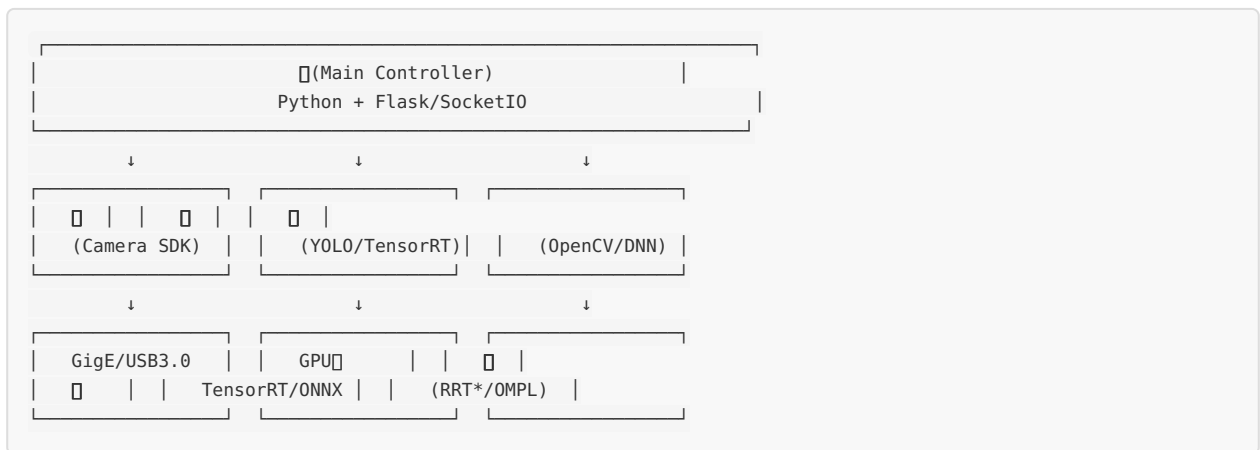


Anti-Interference-2D

Anti-Interference-2D

1.

1.1



1.2

```

steel_detection/
├── config/                                # 
│   ├── camera_config.yaml               # 
│   ├── model_config.yaml                # 
│   └── grasp_config.yaml                 # 
├── models/                               # 
│   ├── yolov8_defect.pt                  # 
│   ├── resnet18_edge.onnx               # 
│   └── grasp_quality.onnx               # 
├── src/                                   # 
│   ├── __init__.py                      # 
│   ├── camera/                          # 
│   │   ├── __init__.py                  # 
│   │   ├── gige_camera.py               # Gige 
│   │   ├── polarized_capture.py         # 
│   │   └── hdr_capture.py               # HDR 
│   ├── detection/                       # 
│   │   ├── __init__.py                  # 
│   │   ├── yolo_detector.py             # YOLO 
│   │   ├── detector_pipeline.py         # 
│   │   └── tensorrt_infer.py            # TensorRT 
│   ├── edge/                            # 
│   │   ├── __init__.py                  # 
│   │   ├── canny_edge.py                # Canny 
│   │   ├── adaptive_edge.py             # 
│   │   └── subpixel_edge.py             # 
│   ├── grasp/                           # 
│   │   ├── __init__.py                  # 
│   │   ├── rrt_planner.py               # RRT 
│   │   ├── grasp_selector.py            # 
│   │   └── path_smoother.py             # 
│   └── utils/                           # 
│       ├── __init__.py                  # 
│       ├── logger.py                    # 
│       ├── monitor.py                   # 
│       └── plc_comm.py                  # PLC 
├── scripts/                              # 
│   ├── train.py                         # 
│   ├── evaluate.py                      # 
│   └── export_model.py                  # 
├── tests/                                # 
│   ├── test_camera.py                   # 
│   ├── test_detection.py                # 
│   └── test_edge.py                     # 
├── requirements.txt                      # 
├── Dockerfile                           # Docker 
└── main.py                              # 

```

2.

2.1

```
# requirements.txt
# PyTorch
torch>=2.0.0
torchvision>=0.15.0

# YOLOv8
ultralytics>=8.0.0

# OpenCV
opencv-python>=4.8.0
opencv-contrib-python>=4.8.0

# 
numpy>=1.24.0
scipy>=1.10.0

# 
Pillow>=9.5.0
imageio>=2.28.0

# 
PyYAML>=6.0

# 
onnxruntime-gpu>=1.15.0
tensorrt>=8.5.0

# Web
flask>=2.3.0
flask-socketio>=5.3.0
python-socketio>=5.8.0

# 
pyzmq>=25.0.0

# PLC
pymodbus>=3.3.0

# 
loguru>=0.7.0

# 
psutil>=5.9.0

# 
pypylon>=2.0.0 # Basler
```

2.2 CUDA□□□□

```
# check_cuda.py
import torch
import sys

def check_cuda_environment():
    """CUDA"""
    print("=" * 50)
    print("CUDA")
    print("=" * 50)

    print(f"\nPyTorch {torch.__version__}")
    print(f"CUDA {torch.cuda.is_available()}")

    if not torch.cuda.is_available():
        print("CUDA")
        sys.exit(1)

    print(f"\nCUDA {torch.version.cuda}")
    print(f"cuDNN {torch.backends.cudnn.version()}")
    print(f"\nGPU {torch.cuda.device_count()}")

    for i in range(torch.cuda.device_count()):
        props = torch.cuda.get_device_properties(i)
        print(f"\nGPU {i}: {props.name}")
        print(f"  {props.major}.{props.minor}")
        print(f"  {props.total_memory / 1024**3:.2f} GB")
        print(f"  {props.multi_processor_count}")

    # GPU
    print("\nGPU.")
    x = torch.randn(1000, 1000).cuda()
    y = torch.matmul(x, x)
    print(f"GPU {y.shape}")

    print("\n" + "=" * 50)
    print("")
    print("=" * 50)

if __name__ == "__main__":
    check_cuda_environment()
```

2.3 Docker

```
# Dockerfile
FROM nvidia/cuda:11.8-cudnn8-runtime-ubuntu22.04

WORKDIR /app

#
RUN apt-get update && apt-get install -y \
    python3.10 \
    python3-pip \
    libgl1-mesa-glx \
    libgl2.0-0 \
    libsm6 \
    libxext6 \
    libxrender-dev \
    libgomp1 \
    && rm -rf /var/lib/apt/lists/*

# Python
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

#
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

#
COPY . .

# CUDA
RUN python -c "import torch; torch.cuda.init()"

#
EXPOSE 5000 5001

#
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:5000/health') || exit 1"

#
CMD ["python3", "main.py"]
```

2.4 □□□

```
# config/system_config.yaml
system:
  name: Anti-Interference-2D
  version: 1.0.0
  log_level: INFO
  log_file: logs/system.log

camera:
  type: GigE
  manufacturer: Basler
  model: acA4096-30uc
  resolution:
    width: 4096
    height: 3000
  fps: 30
  exposure:
    mode: Timed
    time: 10000 # us
  gain:
    mode: Manual
    value: 1.0
  trigger:
    mode: Off # []
    source: Software

polarization:
  enabled: true
  polarizer_angle: 90 # []
  highlight_threshold: 0.7
  suppress_ratio: 0.5

hdr:
  enabled: true
  num_exposures: 5
  exposure_times:
    - 1/500
    - 1/250
    - 1/125
    - 1/60
    - 1/30
  fusion_method: Mertens

detection:
  model_path: models/yolov8_defect.engine
  conf_threshold: 0.5
  iou_threshold: 0.4
  max_detections: 100
  device: cuda:0
  batch_size: 1

edge:
  method: adaptive_canny
  gaussian_ksize: 5
  gaussian_sigma: 1.5
  high_threshold_percentile: 85
  low_threshold_ratio: 0.4
  morph_close_iterations: 2
  min_contour_length: 20
  subpixel_enabled: true

grasp:
  offset_from_edge: 50 # mm
  approach_height: 100 # mm
  max_grasp_force: 500 # N
  min_grasp_force: 100 # N
```

```
plc:
  enabled: true
  ip: 192.168.1.100
  port: 502
  slave_id: 1
  registers:
    defect_status: 40001
    defect_count: 40002
    grasp_command: 40100
    system_ready: 40110

performance:
  target_fps: 30
  max_detection_time_ms: 50
  max_edge_time_ms: 20
  max_planning_time_ms: 500
```

3. □□□□□□

3.1 GigE□□□□□□

```

# src/camera/gige_camera.py
import numpy as np
from pypylon import pylon
import cv2
import time
from typing import Optional, Tuple

class GigeCamera:
    """Gige"""

    def __init__(self, config: dict):
        self.config = config
        self.camera: Optional[pylon.InstantCamera] = None
        self.converter = None
        self.is_open = False

    def open(self):
        """
        #
        tlf = pylon.TlFactory.GetInstance()

        #
        devices = tlf.EnumerateDevices()

        if len(devices) == 0:
            raise RuntimeError("")

        # Gige
        for device in devices:
            if device.GetDeviceClass() == 'BaslerGige':
                self.camera = pylon.InstantCamera(tlf.CreateDevice(device))
                break

        if self.camera is None:
            self.camera = pylon.InstantCamera(tlf.CreateDevice(devices[0]))

        #
        self.camera.Open()

        #
        self._configure_camera()

        #
        self.converter = pylon.ImageFormatConverter()
        self.converter.OutputPixelFormat = pylon.PixelType_Mono8
        self.converter.OutputBitAlignment = pylon.OutputBitAlignment_MsbRight

        self.is_open = True
        print(f" {self.camera.GetDeviceInfo().GetModelName()}")

    def _configure_camera(self):
        """
        #
        nodemap = self.camera.GetNodeMap()

        #
        width = self.config.get('resolution', {}).get('width', 4096)
        height = self.config.get('resolution', {}).get('height', 3000)

        # ROI (Area of Interest)
        pylon.GuiFeatureNode('Width', nodemap).SetValue(width)
        pylon.GuiFeatureNode('Height', nodemap).SetValue(height)

        #
        exposure_time = self.config.get('exposure', {}).get('time', 10000)
        pylon.GuiFeatureNode('ExposureTime', nodemap).SetValue(exposure_time)

```

```

# []
gain = self.config.get('gain', {}).get('value', 1.0)
pylon.GuiFeatureNode('Gain', nodemap).SetValue(gain)

# []
fps = self.config.get('fps', 30)
pylon.GuiFeatureNode('AcquisitionFrameRate', nodemap).SetValue(fps)

# []
trigger_mode = self.config.get('trigger', {}).get('mode', 'Off')
pylon.GuiFeatureNode('TriggerMode', nodemap).SetValue(trigger_mode)

if trigger_mode == 'On':
    trigger_source = self.config.get('trigger', {}).get('source', 'Software')
    pylon.GuiFeatureNode('TriggerSource', nodemap).SetValue(trigger_source)

def grab(self, num_images: int = 1) -> np.ndarray:
    """
    []

    Args:
        num_images: []

    Returns:
        image: []
    """
    if not self.is_open:
        raise RuntimeError("[]")

    # []
    self.camera.StartGrabbing()

    images = []
    grab_result = self.camera.RetrieveResult(
        5000, # []s
        pylon.GrabStrategy_LatestImageOnly
    )

    try:
        if grab_result.GrabSucceeded():
            image = self.converter.Convert(grab_result)
            img_array = image.GetArray()
            images.append(img_array.copy())

        while len(images) < num_images and self.camera.IsGrabbing():
            grab_result = self.camera.RetrieveResult(5000, pylon.GrabStrategy_LatestImageOnly)
            if grab_result.GrabSucceeded():
                image = self.converter.Convert(grab_result)
                images.append(image.GetArray().copy())

    finally:
        grab_result.Release()

    if num_images == 1:
        return images[0] if images else None
    return images

def grab_one(self) -> np.ndarray:
    """[]"""
    return self.grab(num_images=1)

def set_exposure(self, exposure_us: float):
    """[]"""
    if self.is_open:
        nodemap = self.camera.GetNodeMap()
        pylon.GuiFeatureNode('ExposureTime', nodemap).SetValue(exposure_us)

```

```
def trigger(self):
    """
    """
    if self.is_open:
        nodemap = self.camera.GetNodeMap()
        pylon GuiFeatureNode('TriggerSoftware', nodemap).Execute()

def close(self):
    """
    """
    if self.camera and self.is_open:
        self.camera.StopGrabbing()
        self.camera.Close()
        self.is_open = False
        print("")

def __enter__(self):
    self.open()
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.close()
```

3.2 □□□□□□

```

# src/camera/polarized_capture.py
import numpy as np
import cv2
import threading
import time
from typing import Optional, Tuple, List

class PolarizedImage:
    """□"""

    def __init__(self, rgb: np.ndarray, intensity: np.ndarray,
                 degree: np.ndarray, angle: np.ndarray):
        """
        Args:
            rgb: RGB□(H, W, 3)
            intensity: □(H, W)
            degree: □(H, W)
            angle: □(H, W)
        """
        self.rgb = rgb
        self.intensity = intensity
        self.degree = degree
        self.angle = angle
        self.timestamp = time.time()

class PolarizedCamera:
    """□"""

    def __init__(self, camera, config: dict):
        """
        Args:
            camera: GigECamera□
            config: □
        """
        self.camera = camera
        self.config = config
        self.is_running = False
        self.frame_buffer = None
        self.lock = threading.Lock()
        self.thread = None

        # □
        self.polarizer_angles = [0, 45, 90, 135] # □

    def capture_with_polarization(self) -> Optional[PolarizedImage]:
        """
        □

        □
        □

        Returns:
            PolarizedImage: □
        """
        if not self.camera.is_open:
            raise RuntimeError("□")

        # □: □
        # □: □

        # □□
        rgb_images = []
        for angle in self.polarizer_angles:
            # □
            # self.set_polarizer_angle(angle)

```

```

        # □
        time.sleep(0.1)

    # □
    img = self.camera.grab_one()
    rgb_images.append(img)

    # □
    intensity, degree, angle = self._compute_polarization(rgb_images)

    # RGB□
    if len(rgb_images[0].shape) == 2:
        rgb = cv2.cvtColor(rgb_images[0], cv2.COLOR_GRAY2RGB)
    else:
        rgb = rgb_images[0]

    return PolarizedImage(
        rgb=rgb,
        intensity=intensity,
        degree=degree,
        angle=angle
    )

def _compute_polarization(self, images: List[np.ndarray]) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    □

    Stokes□

    Args:
        images: □[I0, I45, I90, I135]

    Returns:
        intensity: □
        degree: □
        angle: □
    """
    # float□
    I0 = images[0].astype(np.float32)
    I45 = images[1].astype(np.float32)
    I90 = images[2].astype(np.float32)
    I135 = images[3].astype(np.float32)

    # Stokes□
    S0 = (I0 + I90) / 2 # □
    S1 = (I0 - I90) / 2 # □□
    S2 = (I45 - I135) / 2 # □

    # □
    intensity = S0

    # □
    with np.errstate(divide='ignore', invalid='ignore'):
        DoLP = np.sqrt(S1**2 + S2**2) / (S0 + 1e-10)
        DoLP = np.clip(DoLP, 0, 1)

    # □(0□□i)
    AoLP = 0.5 * np.arctan2(S2, S1)
    # □
    AoLP_deg = np.rad2deg(AoLP)

    return intensity, DoLP, AoLP_deg

def process_image(self, polarized_img: PolarizedImage,
                  highlight_threshold: float = 0.7,
                  suppress_ratio: float = 0.5) -> np.ndarray:
    """

```

```

    """
    Args:
        polarized_img: np.ndarray
        highlight_threshold: float
        suppress_ratio: float

    Returns:
        processed: np.ndarray
    """
    # Create highlight mask
    highlight_mask = (polarized_img.degree > highlight_threshold).astype(np.float32)

    # Suppress highlights
    suppressed = polarized_img.intensity * (1 - suppress_ratio * highlight_mask)

    # Apply CLAHE
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(suppressed.astype(np.uint8))

    return enhanced

def start_continuous_capture(self):
    """
    Start the continuous capture loop.
    """
    self.is_running = True
    self.thread = threading.Thread(target=self._capture_loop)
    self.thread.daemon = True
    self.thread.start()

def _capture_loop(self):
    """
    The main loop for capturing and processing frames.
    """
    while self.is_running:
        try:
            polarized_img = self.capture_with_polarization()
            processed = self.process_image(polarized_img)

            with self.lock:
                self.frame_buffer = processed.copy()

        except Exception as e:
            print(f"Error: {e}")
            time.sleep(0.1)

def get_frame(self) -> Optional[np.ndarray]:
    """
    Get the current frame from the buffer.
    """
    with self.lock:
        if self.frame_buffer is not None:
            return self.frame_buffer.copy()
    return None

def stop(self):
    """
    Stop the continuous capture loop.
    """
    self.is_running = False
    if self.thread:
        self.thread.join(timeout=2)

```


3.3 HDR□□□

```

# src/camera/hdr_capture.py
import numpy as np
import cv2
import time
from typing import List, Tuple, Optional

class HDRImaging:
    """
    """

    def __init__(self, camera, config: dict):
        """
        Args:
            camera: GigECamera
            config: HDR
        """
        self.camera = camera
        self.config = config
        self.exposure_times = config.get('exposure_times',
                                          [1/500, 1/250, 1/125, 1/60, 1/30])

    def capture_hdr_sequence(self) -> List[Tuple[np.ndarray, float]]:
        """
        HDR

        Returns:
            sequences: [(image, exposure_time), ...]
        """
        sequences = []

        for exp_time in self.exposure_times:
            # 
            exp_us = int(1.0 / exp_time * 1e6) if exp_time < 1 else int(exp_time * 1e6)
            self.camera.set_exposure(exp_us)

            # 
            time.sleep(0.05)

            # 
            img = self.camera.grab_one()
            sequences.append((img, exp_time))

        return sequences

    def merge_mertens(self, sequences: List[Tuple[np.ndarray, float]]) -> np.ndarray:
        """
        Mertens HDR

        Args:
            sequences: [(image, exposure_time), ...]

        Returns:
            hdr: HDR[0, 1]
        """
        images = [seq[0] for seq in sequences]

        # 
        merge = cv2.createMergeMertens()

        # B2
        float_images = []
        for img in images:
            if img.dtype == np.uint8:
                img_float = img.astype(np.float32) / 255.0
            else:
                img_float = img.astype(np.float32)
            float_images.append(img_float)

```

```

# []
merged = merge.process(float_images)

return np.clip(merged, 0, 1)

def merge_debevec(self, sequences: List[Tuple[np.ndarray, float]],
                  gamma: float = 1.0) -> np.ndarray:
    """
    []Debevec HDR

    Args:
        sequences: [(image, exposure_time), ...]
        gamma: gamma[]

    Returns:
        hdr: HDR[]
    """
    images = []
    times = []

    for img, exp_time in sequences:
        if img.dtype == np.uint8:
            images.append(img.astype(np.float32) / 255.0)
        else:
            images.append(img.astype(np.float32))
        times.append(exp_time)

    images = np.array(images)
    times = np.array(times, dtype=np.float32)

    # []
    calibrate = cv2.createCalibrateDebevec()
    response = calibrate.process(images, times)

    # HDR
    merge_debevec = cv2.createMergeDebevec()
    hdr = merge_debevec.process(images, times, response)

    # Gamma[]
    if gamma != 1.0:
        hdr = np.power(hdr, 1.0 / gamma)

    return hdr

def tonemap(self, hdr: np.ndarray,
            method: str = 'Drago') -> np.ndarray:
    """
    HDR[]HDR

    Args:
        hdr: HDR[][0, 1] []
        method: 'Drago', 'Reinhard', 'Mantiuk', 'Ferry'

    Returns:
        ldr:[][0, 255] uint8
    """
    if method == 'Drago':
        tonemap = cv2.createTonemapDrago(1.0, 0.3)
    elif method == 'Reinhard':
        tonemap = cv2.createTonemapReinhard(1.0, 0.5, 0.0, 0.0)
    elif method == 'Mantiuk':
        tonemap = cv2.createTonemapMantiuk(2.2, 0.85, 1.2)
    elif method == 'Ferry':
        tonemap = cv2.createTonemapFerrywal(1.0)
    else:
        raise ValueError(f"[] {method}")

```

```
# []
ldr = tonemap.process(hdr)

# [[0, 255]
ldr = np.clip(ldr * 255, 0, 255).astype(np.uint8)

return ldr

def capture_and_fuse(self) -> Tuple[np.ndarray, np.ndarray]:
    """
    HDR[]

    Returns:
        fused: [[0, 1]
        ldr: [[0, 255]
    """
    # []
    sequences = self.capture_hdr_sequence()

    # Mertens[]
    fused = self.merge_mertens(sequences)

    # []
    ldr = self.tonemap(fused)

    return fused, ldr
```

4.

4.1 YOLOv8

```

# src/detection/yolo_detector.py
import torch
import numpy as np
import cv2
from typing import List, Dict, Optional, Tuple
from ultralytics import YOLO

class DefectDetectionResult:
    """
    """

    def __init__(self, bbox: List[float], conf: float, cls: int, name: str):
        self.bbox = bbox      # [x1, y1, x2, y2]
        self.conf = conf      # 
        self.cls = cls        # 
        self.name = name      # 

    def to_dict(self) -> dict:
        return {
            'bbox': self.bbox,
            'confidence': self.conf,
            'class': self.cls,
            'class_name': self.name
        }

class YOLOv8Detector:
    """YOLOv8"""

    # 
    CLASS_NAMES = [
        'crazing',          # 
        'inclusion',          # 
        'patches',           # 
        'pitted_surface',    # 
        'rolled-in_scale',   # 
        'scratches'          # 
    ]

    def __init__(self, model_path: str, conf_threshold: float = 0.5,
                 iou_threshold: float = 0.4, device: str = 'cuda:0'):
        """
        Args:
            model_path: (.pt or .engine)
            conf_threshold: 
            iou_threshold: IOU
            device: 
        """
        self.model_path = model_path
        self.conf_threshold = conf_threshold
        self.iou_threshold = iou_threshold
        self.device = device

    # 
    if model_path.endswith('.pt'):
        self.model = YOLO(model_path)
        self.model.to(device)
        self.use_trt = False
    elif model_path.endswith('.engine'):
        self.use_trt = True
        # TensorRT
        self.trt_model = self._load_tensorrt(model_path)
    else:
        raise ValueError(f"{model_path}")

    print(f"YOLOv8 {model_path}")

    def _load_tensorrt(self, engine_path: str):

```

```

    """TensorRT"""
    # TensorRT
    # trt_infer.py
    pass

@torch.no_grad()
def detect(self, image: np.ndarray) -> List[DefectDetectionResult]:
    """
    Args:
        image: RGB(H, W, 3), uint8 or float32

    Returns:
        results:
    """
    if self.use_trt:
        return self._detect_trt(image)
    else:
        return self._detect_yolo(image)

def _detect_yolo(self, image: np.ndarray) -> List[DefectDetectionResult]:
    """YOLO"""
    #
    results = self.model.predict(
        image,
        conf=self.conf_threshold,
        iou=self.iou_threshold,
        verbose=False,
        device=self.device
    )

    #
    detections = []
    for result in results:
        boxes = result.boxes
        for box in boxes:
            x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
            conf = float(box.conf[0])
            cls = int(box.cls[0])
            name = self.CLASS_NAMES[cls] if cls < len(self.CLASS_NAMES) else f"class_{cls}"

            detections.append(DefectDetectionResult(
                bbox=[float(x1), float(y1), float(x2), float(y2)],
                conf=conf,
                cls=cls,
                name=name
            ))

    return detections

def _detect_trt(self, image: np.ndarray) -> List[DefectDetectionResult]:
    """TensorRT"""
    # TensorRT
    #
    input_tensor = self._preprocess(image)

    #
    outputs = self.trt_model.inference(input_tensor)

    #
    detections = self._postprocess_trt(outputs)

    return detections

def _preprocess(self, image: np.ndarray) -> torch.Tensor:
    """

```

```

# []
img = cv2.resize(image, (640, 640))

# []
img = img.astype(np.float32) / 255.0

# HWC to CHW
img = np.transpose(img, (2, 0, 1))

# NCHW
img = np.expand_dims(img, 0)

return torch.from_numpy(img).cuda()

def _postprocess_trt(self, outputs: np.ndarray) -> List[DefectDetectionResult]:
    """TensorRT"""
    # YOLO (1, 84, 8400) [obj, x, y, w, h, cls...]
    detections = []

    outputs = outputs[0] # (84, 8400)
    outputs = outputs.T # (8400, 84)

    for det in outputs:
        obj_conf = det[4]
        if obj_conf < self.conf_threshold:
            continue

        cls_confs = det[5:]
        cls_id = np.argmax(cls_confs)
        cls_conf = cls_confs[cls_id]

        if cls_conf < self.conf_threshold:
            continue

        # []
        x, y, w, h = det[:4]
        x1 = x - w / 2
        y1 = y - h / 2
        x2 = x + w / 2
        y2 = y + h / 2

        detections.append(DefectDetectionResult(
            bbox=[float(x1), float(y1), float(x2), float(y2)],
            conf=float(obj_conf * cls_conf),
            cls=int(cls_id),
            name=self.CLASS_NAMES[cls_id]
        ))

    # NMS
    detections = self._nms(detections)

    return detections

def _nms(self, detections: List[DefectDetectionResult],
        iou_threshold: float = 0.4) -> List[DefectDetectionResult]:
    """[]"""
    if not detections:
        return []

    boxes = np.array([d.bbox for d in detections])
    scores = np.array([d.conf for d in detections])

    # xywh[]
    x1 = boxes[:, 0]
    y1 = boxes[:, 1]
    x2 = boxes[:, 2]
    y2 = boxes[:, 3]

```



```

        areas = (x2 - x1) * (y2 - y1)

        # 计算得分排序
        order = scores.argsort()[::-1]

        keep = []
        while order.size > 0:
            i = order[0]
            keep.append(i)

            # 计算当前检测框的边界
            xx1 = np.maximum(x1[i], x1[order[1:]])
            yy1 = np.maximum(y1[i], y1[order[1:]])
            xx2 = np.minimum(x2[i], x2[order[1:]])
            yy2 = np.minimum(y2[i], y2[order[1:]])

            w = np.maximum(0.0, xx2 - xx1)
            h = np.maximum(0.0, yy2 - yy1)
            inter = w * h

            iou = inter / (areas[i] + areas[order[1:]] - inter)

            # 根据IOU阈值筛选
            inds = np.where(iou <= iou_threshold)[0]
            order = order[inds + 1]

        return [detections[i] for i in keep]

def draw_detections(self, image: np.ndarray,
                    detections: List[DefectDetectionResult]) -> np.ndarray:
    """
    result_img = image.copy()

    for det in detections:
        x1, y1, x2, y2 = [int(v) for v in det.bbox]

        # 获取颜色
        color = self._get_class_color(det.cls)

        # 绘制检测框
        cv2.rectangle(result_img, (x1, y1), (x2, y2), color, 2)

        # 添加标签
        label = f"{det.name}: {det.conf:.2f}"
        label_size, _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1)
        cv2.rectangle(result_img, (x1, y1 - label_size[1] - 5),
                      (x1 + label_size[0], y1), color, -1)
        cv2.putText(result_img, label, (x1, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    return result_img

def _get_class_color(self, cls: int) -> Tuple[int, int, int]:
    """
    colors = [
        (255, 0, 0), # 红色
        (0, 255, 0), # 绿色
        (0, 0, 255), # 蓝色
        (255, 255, 0), # 黄色
        (255, 0, 255), # 品红
        (0, 255, 255) # 青色
    ]
    return colors[cls % len(colors)]

```

4.2 TensorRT□□□

```

# src/detection/tensorrt_infer.py
import tensorrt as trt
import torch
import numpy as np
import pycuda.driver as cuda
import pycuda.autoinit

class TensorRTInfer:
    """TensorRT"""

    def __init__(self, engine_path: str, max_batch_size: int = 8):
        """
        Args:
            engine_path: TensorRT
            max_batch_size: batch size
        """
        self.logger = trt.Logger(trt.Logger.WARNING)
        self.runtime = trt.Runtime(self.logger)
        self.engine = self.runtime.deserialize_cuda_engine(engine_path)
        self.context = self.engine.create_execution_context()
        self.max_batch_size = max_batch_size

        # 
        self.input_idx = self.engine.get_binding_index('images')
        self.output_idx = self.engine.get_binding_index('output')

        # 
        self.input_shape = self.engine.get_binding_shape(self.input_idx)
        self.output_shape = self.engine.get_binding_shape(self.output_idx)

        # GPU
        self.input_size = max_batch_size * self._get_size_from_shape(self.input_shape)
        self.output_size = max_batch_size * self._get_size_from_shape(self.output_shape)

        self.d_input = cuda.mem_alloc(self.input_size)
        self.d_output = cuda.mem_alloc(self.output_size)

        # 
        self.stream = cuda.Stream()

    def _get_size_from_shape(self, shape):
        """Tensor"""
        size = 1
        for dim in shape:
            size *= dim
        return size

    def inference(self, inputs: np.ndarray) -> np.ndarray:
        """
        Args:
            inputs: (N, C, H, W), float32

        Returns:
            outputs: 
        """
        batch_size = inputs.shape[0]

        # batch
        self.context.set_binding_shape(self.input_idx, inputs.shape)

        # GPU
        cuda.memcpy_htod_async(self.d_input, inputs, self.stream)

        # 

```

```

        self.context.execute_async_v2(
            bindings=[int(self.d_input), int(self.d_output)],
            stream_handle=self.stream.handle
        )

        # GPU
        output = np.empty(self.output_shape, dtype=np.float32)
        cuda.memcpy_dtoh_async(output, self.d_output, self.stream)

        # CPU
        self.stream.synchronize()

        return output

    def __del__(self):
        """
        if hasattr(self, 'd_input'):
            self.d_input.free()
        if hasattr(self, 'd_output'):
            self.d_output.free()

class TensorRTBuilder:
    """TensorRT"""

    def __init__(self, logger: trt.Logger = None):
        self.logger = logger or trt.Logger(trt.Logger.WARNING)
        self.builder = trt.Builder(self.logger)
        self.network = None
        self.config = None

    def build_engine(self, onnx_path: str, engine_path: str,
                    fp16: bool = True, int8: bool = False):
        """ONNX to TensorRT"""
        # CPU
        network_flags = 1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
        self.network = self.builder.create_network(network_flags)

        # Builder
        self.config = self.builder.create_builder_config()
        self.config.max_workspace_size = 1 << 30 # 1GB

        # FP16
        if fp16 and self.builder.platform_has_fast_fp16:
            self.config.set_flag(trt.BuilderFlag.FP16)

        # INT8
        if int8:
            self.config.set_flag(trt.BuilderFlag.INT8)

        # ONNX
        parser = trt.OnnxParser(self.network, self.logger)
        with open(onnx_path, 'rb') as f:
            parser.parse(f.read())

        # CPU
        engine = self.builder.build_serialized_network(self.network, self.config)

        # CPU
        with open(engine_path, 'wb') as f:
            f.write(engine)

        print(f"TensorRT {engine_path}")

    @staticmethod
    def export_to_onnx(model, output_path: str, input_shape: tuple,
                    dynamic_batch: bool = True):
        """ONNX"""

```

```
dummy_input = torch.randn(*input_shape).cuda()
```

```
torch.onnx.export(  
    model,  
    dummy_input,  
    output_path,  
    export_params=True,  
    opset_version=11,  
    do_constant_folding=True,  
    input_names=['images'],  
    output_names=['output'],  
    dynamic_axes={  
        'images': {0: 'batch'},  
        'output': {0: 'batch'}  
    } if dynamic_batch else None  
)
```

```
print(f"ONNX {output_path}")
```

5. □□□□□

5.1 Canny□□□

```

# src/edge/canny_edge.py
import cv2
import numpy as np
from typing import Tuple, Optional

class CannyEdgeDetector:
    """Canny□"""

    def __init__(self, config: dict):
        """
        Args:
            config: □
        """
        self.gaussian_ksize = config.get('gaussian_ksize', 5)
        self.gaussian_sigma = config.get('gaussian_sigma', 1.5)
        self.high_threshold_percentile = config.get('high_threshold_percentile', 85)
        self.low_threshold_ratio = config.get('low_threshold_ratio', 0.4)

    def detect(self, gray_image: np.ndarray) -> np.ndarray:
        """
        Canny□

        Args:
            gray_image: □(H, W)

        Returns:
            edges: □(H, W)
        """
        # □
        blurred = cv2.GaussianBlur(
            gray_image,
            (self.gaussian_ksize, self.gaussian_ksize),
            self.gaussian_sigma
        )

        # □
        low_th, high_th = self._compute_adaptive_threshold(blurred)

        # Canny□
        edges = cv2.Canny(blurred, low_th, high_th)

        return edges

    def _compute_adaptive_threshold(self, gray_image: np.ndarray) -> Tuple[int, int]:
        """□□□□
        # □
        sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
        magnitude = np.sqrt(sobelx**2 + sobely**2)

        # □
        high_th = np.percentile(magnitude, self.high_threshold_percentile)
        low_th = high_th * self.low_threshold_ratio

        return int(low_th), int(high_th)

    def detect_with_params(self, gray_image: np.ndarray,
                           low_threshold: int,
                           high_threshold: int) -> np.ndarray:
        """□□□□
        blurred = cv2.GaussianBlur(
            gray_image,
            (self.gaussian_ksize, self.gaussian_ksize),
            self.gaussian_sigma
        )

```

```
edges = cv2.Canny(blurred, low_threshold, high_threshold)
return edges
```


5.2 □□□□□□

```

# src/edge/adaptive_edge.py
import cv2
import numpy as np
from typing import Tuple, Optional, List

class AdaptiveEdgeDetector:
    """□"""

    def __init__(self, config: dict):
        self.config = config
        self.gaussian_ksize = config.get('gaussian_ksize', 5)
        self.gaussian_sigma = config.get('gaussian_sigma', 1.5)
        self.morph_close_iterations = config.get('morph_close_iterations', 2)
        self.min_contour_length = config.get('min_contour_length', 20)

    def detect(self, gray_image: np.ndarray,
               defect_regions: List[dict] = None) -> Tuple[np.ndarray, List]:
        """
        Args:
            gray_image: □
            defect_regions: □

        Returns:
            edges: □
            contours: □
        """
        # 1. □
        denoised = cv2.fastNlMeansDenoising(gray_image, h=10)

        # 2. CLAHE□
        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        enhanced = clahe.apply(denoised)

        # 3. □
        edges = self._canny_adaptive(enhanced)

        # 4. □
        edges = self._morphology_close(edges)

        # 5. □
        if defect_regions:
            edges = self._mask_defect_regions(edges, defect_regions)

        # 6. □
        contours, _ = cv2.findContours(
            edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        # 7. □
        filtered_contours = self._filter_contours(contours)

        return edges, filtered_contours

    def _canny_adaptive(self, gray_image: np.ndarray) -> np.ndarray:
        """□anny"""
        # □
        sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
        magnitude = np.sqrt(sobelx**2 + sobely**2)

        # □
        local_mean = cv2.blur(gray_image, (15, 15))
        local_var = cv2.blur((gray_image - local_mean) ** 2, (15, 15))

        # □

```

```

high_th = np.percentile(magnitude, 85)
low_th = high_th * 0.4

# □
variance_threshold = 50
high_th = np.where(local_var > variance_threshold, high_th * 1.2, high_th)

# □
blurred = cv2.GaussianBlur(gray_image,
                           (self.gaussian_ksize, self.gaussian_ksize),
                           self.gaussian_sigma)

# Canny
edges = cv2.Canny(blurred, int(low_th), int(high_th))

return edges

def _morphology_close(self, edges: np.ndarray) -> np.ndarray:
    """□"""
    kernel = np.ones((3, 3), np.uint8)
    closed = cv2.morphologyEx(
        edges,
        cv2.MORPH_CLOSE,
        kernel,
        iterations=self.morph_close_iterations
    )
    return closed

def _mask_defect_regions(self, edges: np.ndarray,
                        defect_regions: List[dict]) -> np.ndarray:
    """□"""
    mask = np.ones_like(edges, dtype=np.uint8)

    for region in defect_regions:
        x1, y1, x2, y2 = [int(v) for v in region['bbox']]
        margin = 20
        x1 = max(0, x1 - margin)
        y1 = max(0, y1 - margin)
        x2 = min(edges.shape[1], x2 + margin)
        y2 = min(edges.shape[0], y2 + margin)
        mask[y1:y2, x1:x2] = 0

    return edges * mask

def _filter_contours(self, contours) -> List:
    """□"""
    filtered = []
    for contour in contours:
        length = cv2.arcLength(contour, True)
        if length >= self.min_contour_length:
            filtered.append(contour)
    return filtered

def analyze_contour(self, contour) -> dict:
    """□"""
    # □
    area = cv2.contourArea(contour)

    # □
    perimeter = cv2.arcLength(contour, True)

    # □
    x, y, w, h = cv2.boundingRect(contour)

    # □
    rect = cv2.minAreaRect(contour)

```

```
#    
circularity = 4 * np.pi * area / (perimeter ** 2) if perimeter > 0 else 0  
  
return {  
    'area': area,  
    'perimeter': perimeter,  
    'bbox': (x, y, w, h),  
    'min_rect': rect,  
    'circularity': circularity  
}
```

5.3 □□□□□□

```

# src/edge/subpixel_edge.py
import cv2
import numpy as np
from typing import List, Tuple, Optional

class SubpixelEdgeDetector:
    """
    """

    def __init__(self):
        pass

    def refine_corners(self, corners: np.ndarray,
                      gray_image: np.ndarray,
                      window_size: int = 5) -> List:
        """
        Harris
        """

        Args:
            corners: (N, 2)
            gray_image:
            window_size:

        Returns:
            refined:
        """
        if len(corners) == 0:
            return []

        corners_float = corners.astype(np.float32)

        #
        criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_COUNT, 30, 0.1)
        refined = cv2.cornerSubPix(
            gray_image,
            corners_float,
            (window_size, window_size),
            (-1, -1),
            criteria
        )

        return refined.tolist() if refined is not None else corners.tolist()

    def refine_edges(self, edge_points: List[Tuple[int, int]],
                    gray_image: np.ndarray) -> List[Tuple[float, float]]:
        """
        """

        Args:
            edge_points: [(x, y), ...]
            gray_image:

        Returns:
            refined: [(x, y), ...]
        """
        if not edge_points:
            return []

        refined = []

        for y, x in edge_points:
            #
            refined_point = self._parabolic_fitting(y, x, gray_image)
            refined.append(refined_point)

        return refined

```

```

def _parabolic_fitting(self, y: int, x: int,
                      gray_image: np.ndarray) -> Tuple[float, float]:
    """
    []

    []

    Args:
        y, x: []
        gray_image: []

    Returns:
        sub_x, sub_y: []
    """
    h, w = gray_image.shape

    # []
    if y > 0 and y < h - 1:
        i0 = int(gray_image[y-1, x])
        i1 = int(gray_image[y, x])
        i2 = int(gray_image[y+1, x])

        if i0 + i2 - 2*i1 != 0:
            delta = 0.5 * (i0 - i2) / (i0 + i2 - 2*i1)
        else:
            delta = 0
    else:
        delta = 0

    return (float(x), float(y) + delta)

def edge_line_fitting(self, edge_points: List[Tuple[float, float]]
                    ) -> Optional[dict]:
    """
    []

    Args:
        edge_points: []

    Returns:
        line_params: [{'angle', 'distance', 'point'}]
    """
    if len(edge_points) < 2:
        return None

    points = np.array(edge_points, dtype=np.float32)

    # OpenCV fitLine
    try:
        [vx, vy, x0, y0] = cv2.fitLine(
            points,
            cv2.DIST_L2,
            0, # []
            0.01, 0.01 # []
        )

        # []
        angle = np.arctan2(vy, vx)[0] * 180 / np.pi
        cos_a = np.cos(np.arctan2(vy, vx)[0])
        sin_a = np.sin(np.arctan2(vy, vx)[0])
        distance = abs(x0 * sin_a - y0 * cos_a)

    return {
        'angle': float(angle),
        'distance': float(distance),
        'point': (float(x0), float(y0)),
        'direction': (float(vx), float(vy))
    }

```

```
    }
    except:
        return None

def detect_edge_centerline(self, binary_edge: np.ndarray) -> List:
    """
    []

    Args:
        binary_edge: []

    Returns:
        centerline_points: []
    """
    # []
    skeleton = self._skeletonize(binary_edge)

    # []
    points = np.where(skeleton > 0)
    centerline_points = list(zip(points[1], points[0])) # (x, y)

    # []
    refined = self.refine_edges(centerline_points, binary_edge)

    return refined

def _skeletonize(self, binary_image: np.ndarray) -> np.ndarray:
    """
    []
    """
    skeleton = np.zeros_like(binary_image)
    element = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))

    temp = binary_image.copy()
    while True:
        eroded = cv2.erode(temp, element)
        opened = cv2.dilate(eroded, element)
        temp_diff = cv2.subtract(temp, opened)
        skeleton = cv2.bitwise_or(skeleton, temp_diff)
        temp = eroded.copy()

        if cv2.countNonZero(temp) == 0:
            break

    return skeleton
```


6. □□□□□□

6.1 RRT*□□□

```

# src/grasp/rrt_planner.py
import numpy as np
import random
from typing import List, Tuple, Optional, Callable
import heapq

class RRTStarNode:
    """RRT* node"""

    def __init__(self, state: np.ndarray, parent: 'RRTStarNode' = None):
        self.state = state
        self.parent = parent
        self.cost = 0.0 if parent is None else parent.cost + np.linalg.norm(state - parent.state)

class RRTStarPlanner:
    """RRT* planner"""

    def __init__(self, workspace_bounds: List[Tuple[float, float]],
                 step_size: float = 20.0,
                 max_iterations: int = 5000,
                 rewire_radius: float = 50.0,
                 goal_bias: float = 0.1):
        """
        Args:
            workspace_bounds: List of (x_min, x_max), (y_min, y_max), (z_min, z_max)
            step_size: float
            max_iterations: int
            rewire_radius: float
            goal_bias: float
        """
        self.workspace_bounds = workspace_bounds
        self.step_size = step_size
        self.max_iterations = max_iterations
        self.rewire_radius = rewire_radius
        self.goal_bias = goal_bias

        self.nodes: List[RRTStarNode] = []
        self.node_index_map = {} # Dict

    def plan(self, start: List[float], goal: List[float],
            obstacle_check_fn: Optional[Callable] = None) -> Optional[List[List[float]]]:
        """
        Args:
            start: [x, y, z]
            goal: [x, y, z]
            obstacle_check_fn: Callable

        Returns:
            path: List[List[float]] or None
        """
        start = np.array(start)
        goal = np.array(goal)

        # Start node
        start_node = RRTStarNode(start)
        self.nodes = [start_node]
        self.node_index_map = {tuple(start): 0}

        for i in range(self.max_iterations):
            # Sample
            if random.random() < self.goal_bias:
                random_state = goal.copy()
            else:
                random_state = self._sample_state()

```

```

        # □
        nearest_idx, nearest_node = self._find_nearest(random_state)

        # □
        new_node = self._steer(nearest_node, random_state)

        # □
        if obstacle_check_fn is not None and obstacle_check_fn(new_node.state):
            continue

        # □
        best_parent, best_idx = self._find_best_parent(new_node, obstacle_check_fn)

        if best_parent is None:
            continue

        new_node.parent = best_parent
        new_node.cost = best_parent.cost + np.linalg.norm(new_node.state - best_parent.state)

        new_idx = len(self.nodes)
        self.nodes.append(new_node)
        self.node_index_map[tuple(new_node.state.round(2))] = new_idx

        # □
        self._rewire(new_node, new_idx, obstacle_check_fn)

        # □
        if np.linalg.norm(new_node.state - goal) < self.step_size:
            return self._extract_path(new_node)

    return None

def _sample_state(self) -> np.ndarray:
    """□□□□
    state = []
    for low, high in self.workspace_bounds:
        state.append(random.uniform(low, high))
    return np.array(state)

def _find_nearest(self, state: np.ndarray) -> Tuple[int, RRTStarNode]:
    """□□□□
    min_dist = float('inf')
    nearest_idx = None
    nearest = None

    for i, node in enumerate(self.nodes):
        dist = np.linalg.norm(node.state - state)
        if dist < min_dist:
            min_dist = dist
            nearest_idx = i
            nearest = node

    return nearest_idx, nearest

def _steer(self, from_node: RRTStarNode, to_state: np.ndarray) -> RRTStarNode:
    """□□□□
    direction = to_state - from_node.state
    distance = np.linalg.norm(direction)

    if distance <= self.step_size:
        new_state = to_state
    else:
        new_state = from_node.state + (direction / distance) * self.step_size

    return RRTStarNode(new_state)

```

```

def _find_best_parent(self, new_node: RRTStarNode,
                      obstacle_check_fn: Optional[Callable]) -> Tuple[Optional[RRTStarNode], Optional[int]]:
    """□□□□
    best_parent = None
    best_cost = float('inf')
    best_idx = None

    for i, node in enumerate(self.nodes):
        dist = np.linalg.norm(new_node.state - node.state)
        if dist > self.step_size:
            continue

        # □
        if obstacle_check_fn is not None:
            if not self._is_edge_valid(node.state, new_node.state, obstacle_check_fn):
                continue

        cost = node.cost + dist
        if cost < best_cost:
            best_cost = cost
            best_parent = node
            best_idx = i

    return best_parent, best_idx

def _is_edge_valid(self, state1: np.ndarray, state2: np.ndarray,
                  obstacle_check_fn: Callable) -> bool:
    """□□□□
    num_checks = 10
    for t in np.linspace(0, 1, num_checks):
        point = (1 - t) * state1 + t * state2
        if obstacle_check_fn(point):
            return False
    return True

def _rewire(self, new_node: RRTStarNode, new_idx: int,
            obstacle_check_fn: Optional[Callable]):
    """□□□□
    for i, node in enumerate(self.nodes):
        if i == new_idx:
            continue

        dist = np.linalg.norm(new_node.state - node.state)
        if dist > self.rewire_radius:
            continue

        if not self._is_edge_valid(new_node.state, node.state, obstacle_check_fn):
            continue

        new_cost = new_node.cost + dist
        if new_cost < node.cost:
            node.parent = new_node
            node.cost = new_cost

        # □
        self._update_descendant_costs(node, new_node.cost - node.cost)

def _update_descendant_costs(self, node: RRTStarNode, cost_delta: float):
    """□□□□
    # BFS□
    queue = [node]
    visited = set()

    while queue:
        current = queue.pop(0)
        if id(current) in visited:
            continue

```

```
        visited.add(id(current))

    for child in self.nodes:
        if child.parent == current:
            child.cost += cost_delta
            queue.append(child)

def _extract_path(self, end_node: RRTStarNode) -> List[List[float]]:
    """
    """
    path = []
    node = end_node
    while node is not None:
        path.append(node.state.tolist())
        node = node.parent
    path.reverse()
    return path
```

6.2 □□□□

```

# src/grasp/path_smoother.py
import numpy as np
from scipy.interpolate import splprep, splev
from typing import List, Tuple

class PathSmoother:
    """
    """

    def __init__(self):
        pass

    def b_spline_smooth(self, path: List[List[float]],
                        num_points: int = None) -> List[List[float]]:
        """
        B-spline smoothing.

        Args:
            path: List of points.
            num_points: Number of points to smooth to.

        Returns:
            smoothed: Smoothed path.
        """
        if len(path) < 4:
            return path

        path = np.array(path)
        if num_points is None:
            num_points = len(path) * 3

        # Time
        t = np.linspace(0, 1, len(path))
        t_smooth = np.linspace(0, 1, num_points)

        # Smoothed path
        smoothed = np.zeros((num_points, path.shape[1]))
        for i in range(path.shape[1]):
            k = 3 if len(path) > 3 else len(path) - 1
            tck, _ = splprep([path[:, i]], t, s=0, k=k)
            smoothed[:, i] = splev(t_smooth, tck)[0]

        # Interpolate
        smoothed[:, -1] = np.interp(t_smooth, t, path[:, -1])

        return smoothed.tolist()

    def gradient_descent_smooth(self, path: List[List[float]],
                                obstacles: List = None,
                                iterations: int = 100,
                                step_size: float = 0.1) -> List[List[float]]:
        """
        Gradient descent smoothing.

        Args:
            path: List of points.
            obstacles: List of obstacles.
            iterations: Number of iterations.
            step_size: Step size.

        Returns:
            smoothed: Smoothed path.
        """
        smoothed = np.array(path, dtype=np.float64)

        for _ in range(iterations):
            for i in range(1, len(smoothed) - 1):

```

```

        # []
        smooth_grad = smoothed[i-1] + smoothed[i+1] - 2 * smoothed[i]

    # []
    if obstacles:
        obs_grad = self._obstacle_gradient(smoothed[i], obstacles)
    else:
        obs_grad = 0

    # []
    smoothed[i] += step_size * (smooth_grad - 0.1 * obs_grad)

    return smoothed.tolist()

def _obstacle_gradient(self, point: np.ndarray,
                       obstacles: List, epsilon: float = 1e-6) -> np.ndarray:
    """[]"""
    grad = np.zeros_like(point)
    for obs in obstacles:
        if 'center' in obs:
            center = np.array(obs['center'])
            radius = obs.get('radius', 50)
            dist = np.linalg.norm(point - center)
            if dist < radius * 2:
                grad += (point - center) / (dist + epsilon)
    return grad

def shortcut_smoothing(self, path: List[List[float]],
                      obstacles: List = None,
                      iterations: int = 50) -> List[List[float]]:
    """
    []

    Args:
        path: []
        obstacles: []
        iterations: []

    Returns:
        smoothed: []
    """
    smoothed = [p.copy() for p in path]

    for _ in range(iterations):
        if len(smoothed) <= 2:
            break

        # []
        i = random.randint(0, len(smoothed) - 2)
        j = random.randint(i + 2, len(smoothed) - 1)

        # []
        if obstacles is None or self._is_shortcut_valid(smoothed[i], smoothed[j], obstacles):
            # []
            smoothed = smoothed[:i+1] + smoothed[j:]

    return smoothed

def _is_shortcut_valid(self, p1: List[float], p2: List[float],
                      obstacles: List) -> bool:
    """[]"""
    num_checks = 10
    p1 = np.array(p1)
    p2 = np.array(p2)

    for t in np.linspace(0, 1, num_checks):
        point = (1 - t) * p1 + t * p2

```



```
    for obs in obstacles:
        if 'center' in obs:
            center = np.array(obs['center'])
            radius = obs.get('radius', 50)
            if np.linalg.norm(point - center) < radius:
                return False
    return True
```

7. Web□□□□

7.1 Flask□□

```

# src/utils/web_service.py
from flask import Flask, request, jsonify, send_file
import cv2
import numpy as np
import base64
import io
import time
from loguru import logger
import threading
from queue import Queue

app = Flask(__name__)

#
detector = None
edge_detector = None
frame_queue = Queue(maxsize=10)
result_queue = Queue(maxsize=10)

def init_app(detector_instance, edge_detector_instance):
    """
    global detector, edge_detector
    detector = detector_instance
    edge_detector = edge_detector_instance

@app.route('/health', methods=['GET'])
def health_check():
    """
    return jsonify({'status': 'ok', 'timestamp': time.time()})

@app.route('/api/detect', methods=['POST'])
def detect_defects():
    """API"""
    try:
        #
        data = request.json
        img_str = data.get('image')
        if not img_str:
            return jsonify({'success': False, 'error': 'No image provided'})

        img_data = base64.b64decode(img_str)
        nparr = np.frombuffer(img_data, np.uint8)
        image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        if image is None:
            return jsonify({'success': False, 'error': 'Invalid image'})

        #
        start_time = time.time()
        detections = detector.detect(image)
        elapsed = (time.time() - start_time) * 1000

        #
        results = []
        for det in detections:
            results.append({
                'bbox': det.bbox,
                'confidence': det.conf,
                'class': det.cls,
                'class_name': det.name
            })

        return jsonify({
            'success': True,
            'detections': results,
            'count': len(results),

```

```

        'processing_time_ms': round(elapsed, 2)
    })

except Exception as e:
    logger.error(f"❌ {e}")
    return jsonify({'success': False, 'error': str(e)})

@app.route('/api/edge', methods=['POST'])
def detect_edges():
    """API"""
    try:
        data = request.json
        img_str = data.get('image')
        if not img_str:
            return jsonify({'success': False, 'error': 'No image provided'})

        img_data = base64.b64decode(img_str)
        nparr = np.frombuffer(img_data, np.uint8)
        image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

        # □
        start_time = time.time()
        edges, contours = edge_detector.detect(gray)
        elapsed = (time.time() - start_time) * 1000

        # □
        _, buffer = cv2.imencode('.png', edges)
        edge_str = base64.b64encode(buffer).decode()

        return jsonify({
            'success': True,
            'edge_image': edge_str,
            'contour_count': len(contours),
            'processing_time_ms': round(elapsed, 2)
        })

    except Exception as e:
        logger.error(f"❌ {e}")
        return jsonify({'success': False, 'error': str(e)})

@app.route('/api/detect_image', methods=['POST'])
def detect_with_image():
    """API"""
    try:
        files = request.files
        if 'image' not in files:
            return jsonify({'success': False, 'error': 'No image file'})

        file = files['image']
        image = cv2.imdecode(
            np.frombuffer(file.read(), np.uint8),
            cv2.IMREAD_COLOR
        )

        # □
        detections = detector.detect(image)

        # □
        result_img = detector.draw_detections(image, detections)

        # □
        _, buffer = cv2.imencode('.jpg', result_img, [cv2.IMWRITE_JPEG_QUALITY, 85])
        result_str = base64.b64encode(buffer).decode()

        return jsonify({
            'success': True,

```

```
        'image': result_str,
        'count': len(detections)
    })

    except Exception as e:
        logger.error(f"{e}")
        return jsonify({'success': False, 'error': str(e)})

def run_server(host='0.0.0.0', port=5000, debug=False):
    """
    """
    logger.info(f"flask {host}:{port}")
    app.run(host=host, port=port, debug=debug, threaded=True)
```

8. PLC□□□□

8.1 Modbus TCP□□


```

        logger.error(f"{e}")
        return False

def write_registers(self, address: int, values: list) -> bool:
    """
    if not self.connected:
        return False

    try:
        self.client.write_registers(address, values, unit=self.slave_id)
        return True
    except ModbusException as e:
        logger.error(f"{e}")
        return False

def read_defect_status(self, registers: dict) -> Optional[Dict[str, Any]]:
    """
    status_base = registers.get('defect_status', 40001)
    count_base = registers.get('defect_count', 40002)

    status = self.read_holding_registers(status_base, 10)
    if status:
        return {
            'defect_count': status[0],
            'edge_quality': status[1],
            'system_status': status[2],
            'error_code': status[3]
        }
    return None

def write_grasp_command(self, x: float, y: float, z: float,
                        angle: float, registers: dict) -> bool:
    """
    cmd_base = registers.get('grasp_command', 40100)

    #
    x_int = int(x * 10) # mm * 10
    y_int = int(y * 10)
    z_int = int(z * 10)
    angle_int = int(angle * 10)

    #
    success = True
    success = success and self.write_register(cmd_base, x_int)
    success = success and self.write_register(cmd_base + 1, y_int)
    success = success and self.write_register(cmd_base + 2, z_int)
    success = success and self.write_register(cmd_base + 3, angle_int)
    success = success and self.write_register(cmd_base + 4, 1) #

    return success

def set_system_ready(self, ready: bool, registers: dict) -> bool:
    """
    ready_reg = registers.get('system_ready', 40110)
    return self.write_register(ready_reg, 1 if ready else 0)

def __enter__(self):
    self.connect()
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.disconnect()

```


9.

9.1

```
# src/utils/logger.py
from loguru import logger
import sys
from pathlib import Path
import rotation

def setup_logger(log_file: str = 'logs/system.log',
                 rotation_size: str = '10 MB',
                 retention: str = '7 days',
                 level: str = 'INFO'):
    """
    # handler
    logger.remove()

    #
    logger.add(
        sys.stdout,
        format='<green>{time:YYYY-MM-DD HH:mm:ss}</green> | <level>{level: <8}</level> | <cyan>{name}</cyan>:<cyan>{function}</cy
        level=level,
        colorize=True
    )

    #
    log_path = Path(log_file)
    log_path.parent.mkdir(parents=True, exist_ok=True)

    logger.add(
        log_file,
        format='{time:YYYY-MM-DD HH:mm:ss} | {level: <8} | {name}:{function}:{line} - {message}',
        level=level,
        rotation=rotation_size,
        retention=retention,
        compression='zip',
        backtrace=True,
        diagnose=True
    )

    return logger

#
# from src.utils.logger import setup_logger
# logger = setup_logger()
```

9.2 □□□

```

# src/utils/monitor.py
import psutil
import time
import torch
from typing import Dict, Any
from dataclasses import dataclass, asdict
from loguru import logger

@dataclass
class PerformanceMetrics:
    """
    fps: float = 0.0
    cpu_percent: float = 0.0
    memory_used_gb: float = 0.0
    memory_percent: float = 0.0
    gpu_memory_used_gb: float = 0.0
    gpu_utilization: float = 0.0
    detection_time_ms: float = 0.0
    edge_time_ms: float = 0.0
    total_time_ms: float = 0.0
    """

class PerformanceMonitor:
    """
    """

    def __init__(self):
        self.metrics = PerformanceMetrics()
        self.frame_count = 0
        self.start_time = time.time()
        self.last_time = time.time()

    def update(self, detection_time_ms: float = 0, edge_time_ms: float = 0):
        """
        """
        current_time = time.time()

        # FPS
        self.frame_count += 1
        elapsed = current_time - self.start_time
        if elapsed > 0:
            self.metrics.fps = self.frame_count / elapsed

        # CPU
        self.metrics.cpu_percent = psutil.cpu_percent(interval=0)
        mem = psutil.virtual_memory()
        self.metrics.memory_used_gb = mem.used / 1024**3
        self.metrics.memory_percent = mem.percent

        # GPU
        if torch.cuda.is_available():
            self.metrics.gpu_memory_used_gb = torch.cuda.memory_allocated() / 1024**3
            self.metrics.gpu_utilization = torch.cuda.utilization()

        # 
        self.metrics.detection_time_ms = detection_time_ms
        self.metrics.edge_time_ms = edge_time_ms
        self.metrics.total_time_ms = detection_time_ms + edge_time_ms

        self.last_time = current_time

    def get_metrics(self) -> Dict[str, Any]:
        """
        """
        return asdict(self.metrics)

    def log_metrics(self):
        """
        """
        m = self.metrics
        logger.debug(

```

```
f"FPS: {m.fps:.1f} | "  
f"CPU: {m.cpu_percent:.1f}% | "  
f"Mem: {m.memory_used_gb:.2f}GB ({m.memory_percent:.1f}%) | "  
f"GPU Mem: {m.gpu_memory_used_gb:.2f}GB | "  
f"Detect: {m.detection_time_ms:.1f}ms | "  
f"Edge: {m.edge_time_ms:.1f}ms"  
)  
  
def reset(self):  
    """  
    self.frame_count = 0  
    self.start_time = time.time()
```

10. □□□□

10.1 □□□

```

# main.py
import argparse
import yaml
from pathlib import Path
from loguru import logger

from src.camera.gige_camera import GigECamera
from src.camera.polarized_capture import PolarizedCamera
from src.detection.yolo_detector import YOLODetector
from src.edge.adaptive_edge import AdaptiveEdgeDetector
from src.grasp.rrt_planner import RRTStarPlanner
from src.grasp.path_smoother import PathSmoother
from src.utils.logger import setup_logger
from src.utils.monitor import PerformanceMonitor
from src.utils.plc_comm import PLCCommunicator
from src.utils.web_service import run_server
import threading

class SteelDetectionSystem:
    """
    """

    def __init__(self, config_path: str):
        #
        with open(config_path, 'r', encoding='utf-8') as f:
            self.config = yaml.safe_load(f)

        #
        self.camera = None
        self.polarized_camera = None
        self.detector = None
        self.edge_detector = None
        self.planner = None
        self.smoother = None
        self.plc = None
        self.monitor = PerformanceMonitor()

    def initialize(self):
        """
        """
        logger.info("=" * 50)
        logger.info("...\n")
        logger.info("=" * 50)

        #
        if self.config.get('camera', {}).get('type') == 'GigE':
            self.camera = GigECamera(self.config['camera'])
            self.camera.open()

            if self.config.get('polarization', {}).get('enabled'):
                self.polarized_camera = PolarizedCamera(
                    self.camera,
                    self.config['polarization']
                )

        #
        self.detector = YOLODetector(
            model_path=self.config['detection']['model_path'],
            conf_threshold=self.config['detection']['conf_threshold'],
            iou_threshold=self.config['detection']['iou_threshold'],
            device=self.config['detection'].get('device', 'cuda:0')
        )

        #
        self.edge_detector = AdaptiveEdgeDetector(self.config['edge'])

        #
        workspace = self.config.get('grasp', {}).get('workspace_bounds',

```

```

[[0, 1000], [0, 1000], [0, 500]])

self.planner = RRTStarPlanner(
    workspace_bounds=workspace,
    step_size=20,
    max_iterations=5000
)
self.smoother = PathSmoother()

# PLC
if self.config.get('plc', {}).get('enabled'):
    self.plc = PLCCommunicator(
        ip=self.config['plc']['ip'],
        port=self.config['plc']['port']
    )
    self.plc.connect()

logger.info("")

def process_frame(self, image):
    """
    import time
    start = time.time()

    # 
    det_start = time.time()
    detections = self.detector.detect(image)
    det_time = (time.time() - det_start) * 1000

    # 
    edge_start = time.time()
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    edges, contours = self.edge_detector.detect(gray)
    edge_time = (time.time() - edge_start) * 1000

    # 
    self.monitor.update(det_time, edge_time)

    return {
        'detections': detections,
        'edges': edges,
        'contours': contours,
        'detection_time_ms': det_time,
        'edge_time_ms': edge_time
    }

def run(self):
    """
    logger.info("")

    try:
        while True:
            # 
            image = self.camera.grab_one()

            # 
            result = self.process_frame(image)

            # 
            self.monitor.log_metrics()

        except KeyboardInterrupt:
            logger.info("")
        finally:
            self.shutdown()

def shutdown(self):
    """

```

```

        logger.info("□..")

    if self.camera:
        self.camera.close()
    if self.plc:
        self.plc.disconnect()

    logger.info("□")

def main():
    parser = argparse.ArgumentParser(description='Anti-Interference-2D□')
    parser.add_argument('--config', type=str, default='config/system_config.yaml',
                        help='□')
    parser.add_argument('--web', action='store_true',
                        help='Web□')
    parser.add_argument('--host', type=str, default='0.0.0.0',
                        help='Web□')
    parser.add_argument('--port', type=int, default=5000,
                        help='Web□')

    args = parser.parse_args()

    # □
    setup_logger()

    # □
    system = SteelDetectionSystem(args.config)

    if args.web:
        # Web□
        from src.utils.web_service import init_app, run_server
        init_app(system.detector, system.edge_detector)
        run_server(host=args.host, port=args.port)
    else:
        # □
        system.initialize()
        system.run()

if __name__ == '__main__':
    main()

```


11. □□□□□□

11.1 □□□□

```

# scripts/download_dataset.py
import os
import requests
import zipfile
from pathlib import Path
from typing import Dict, Optional
from loguru import logger

class DatasetDownloader:
    """□"""

    DATASETS: Dict[str, Dict] = {
        'NEU-DET': {
            'url': 'https://github.com/lxmic/Mamba-YOLO/releases/download/v0.0.1/NEU-DET.zip',
            'path': 'datasets/NEU-DET',
            'description': '□'
        },
        'GC10-DET': {
            'url': 'https://github.com/lxmic/Mamba-YOLO/releases/download/v0.0.1/GC10-DET.zip',
            'path': 'datasets/GC10-DET',
            'description': 'GC10□'
        }
    }

    def __init__(self, save_dir: str = 'datasets'):
        self.save_dir = Path(save_dir)
        self.save_dir.mkdir(parents=True, exist_ok=True)

    def download(self, name: str, force: bool = False) -> Optional[Path]:
        """
        □

        Args:
            name: □
            force: □

        Returns:
            □□None
        """
        if name not in self.DATASETS:
            logger.error(f"□ {name}")
            return None

        info = self.DATASETS[name]
        save_path = self.save_dir / info['path']

        if save_path.exists() and not force:
            logger.info(f"□ {save_path}")
            return save_path

        # □
        zip_path = self.save_dir / f"{name}.zip"
        logger.info(f"□{name}...")

        try:
            response = requests.get(info['url'], stream=True, timeout=300)
            response.raise_for_status()

            total_size = int(response.headers.get('content-length', 0))
            downloaded = 0

            with open(zip_path, 'wb') as f:
                for chunk in response.iter_content(chunk_size=8192):
                    if chunk:
                        f.write(chunk)
                        downloaded += len(chunk)

```

```

        if total_size > 0:
            progress = (downloaded / total_size) * 100
            if downloaded % (1024 * 1024) == 0: # MB
                logger.info(f"{progress:.1f}%")

        # 
        logger.info(f"[{name}...]")
        with zipfile.ZipFile(zip_path, 'r') as zip_ref:
            zip_ref.extractall(self.save_dir)

        # Zip
        os.remove(zip_path)

        logger.info(f"[{save_path}]")
        return save_path

    except Exception as e:
        logger.error(f"[{e}]")
        if zip_path.exists():
            os.remove(zip_path)
        return None

    def download_all(self):
        """
        for name in self.DATASETS:
            self.download(name)

if __name__ == '__main__':
    downloader = DatasetDownloader()
    downloader.download_all()

```

12. □□□□□

12.1 YOLOv8□□

```

# scripts/train.py
import argparse
from ultralytics import YOLO
from pathlib import Path
from loguru import logger

def train_yolov8(data_yaml: str,
                 model_size: str = 'm',
                 epochs: int = 300,
                 batch_size: int = 16,
                 img_size: int = 640,
                 device: str = '0'):
    """
    YOLOv8

    Args:
        data_yaml:
        model_size: (n/s/m/l/x)
        epochs:
        batch_size:
        img_size:
        device:
    """
    logger.info("=" * 50)
    logger.info("YOLOv8")
    logger.info("=" * 50)

    #
    model = YOLO(f'yolov8{model_size}.pt')

    #
    results = model.train(
        data=data_yaml,
        epochs=epochs,
        batch=batch_size,
        imgsz=img_size,
        device=device,
        project='runs/detect',
        name='steel_defect',
        exist_ok=True,
        verbose=True,
        #
        optimizer='AdamW',
        lr0=0.001,
        lrf=0.01,
        momentum=0.937,
        weight_decay=0.0005,
        #
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
        degrees=10,
        translate=0.1,
        scale=0.5,
        flipplr=0.5,
        mosaic=1.0,
        mixup=0.1,
        #
        patience=50,
        close_mosaic=10,
    )

    #
    best_model_path = f"runs/detect/steel_defect/weights/best.pt"
    logger.info(f" {best_model_path}")

```

```
    return results

def main():
    parser = argparse.ArgumentParser(description='YOLOv8')
    parser.add_argument('--data', type=str, required=True,
                        help='')
    parser.add_argument('--model', type=str, default='m',
                        choices=['n', 's', 'm', 'l', 'x'],
                        help='')
    parser.add_argument('--epochs', type=int, default=300,
                        help='')
    parser.add_argument('--batch', type=int, default=16,
                        help='')
    parser.add_argument('--img-size', type=int, default=640,
                        help='')
    parser.add_argument('--device', type=str, default='0',
                        help='')

    args = parser.parse_args()

    train_yolov8(
        data_yaml=args.data,
        model_size=args.model,
        epochs=args.epochs,
        batch_size=args.batch,
        img_size=args.img_size,
        device=args.device
    )

if __name__ == '__main__':
    main()
```

YOLOv1.0

2026-04-27

Anti-Interference-2D